

Informe Técnico General

Por: (Esteban Prieto / Lider de Soporte)

Resumen

- **Proyecto:** DASHKPIs (Frontend: React + Vite + TS + Tailwind; Backend: Django + DRF; BD: SQL Server Azure / SQLite local).
- **Objetivo cumplido:** Personalización de disposición del dashboard, formato de fechas/moneda y preferencias de notificaciones.
- Se resolvieron bloqueos de migraciones, validaciones y comunicación FE/BE para persistir preferencias de usuario.

Backend (Django + DRF)

Hechos clave

UC-04 Configurar disposición de paneles

- Modelo PanelLayoutPreference con constraint única (usuario+proyecto) y uso de safe_default_panel_state.
- Endpoint GET/POST/DELETE /api/dashboard/layout/ para consultar, guardar y restablecer layout.
- Serializer valida paneles y tamaños permitidos; notifica errores con mensajes claros.

UC-05 Configurar formato de fechas y moneda

- Modelo UserFormatPreference (campos formato_fecha, codigo_moneda).
- Endpoint GET/PUT/DELETE /api/usuarios/{id}/preferencias/formato/.
- Migración + script SQL para crear dashboard_user_format_prefs; manejo de defaults si no existe registro.

UC-06 Configurar notificaciones preferidas

- Vista NotificationConfigView devuelve catálogo de canales/tipos con metadatos, lectura y restablecimiento.
- PUT /api/usuarios/{id}/notificaciones/config/ hace upsert en config_notificaciones.

- Pruebas unitarias cubren GET/PUT/DELETE y el cursor manual sobre SQL Server.

Checklist de errores relevantes (Backend)

- Archivo apps/projects/migrations/__init__.py con texto “***” → reparado para permitir migraciones.
- PanelLayoutPreference arrojaba error de unicidad → se deshabilitó el validador duplicado y se mantiene constraint en BD.
- Falta de tablas en SQL Server (layouts, formatos) → scripts ejecutados y migraciones marcadas con --fake-initial.
- Endpoints de preferencias devolvían 500 sin tabla → ahora responden con defaults y manejo de errores.

Pendientes sugeridos (Backend)

- Endpoint GET /api/dashboard/layout/ podría soportar id_proyecto distintos a 0.
- Validar catálogos dinámicos para canales/tipos de notificación desde BD (no hardcoded).
- Agregar logging/auditoría cuando se restablecen o borran preferencias.

Frontend (React + Vite + TS)

Hechos clave

UC-04

- DashboardHome integra botón “Configurar paneles”. Modal permite presets, drag & drop, fijar paneles y restablecer.
- Layout aplicado persiste tras refrescar usando respuesta del backend.

UC-05

- En Configuración, tarjeta “Formatos de fechas y moneda” muestra vista previa y mensajes de confirmación/error.
- Manejo de restablecer y cancelación desesperada. Feedback verde/rojo según resultado del endpoint.

UC-06

- Sección de notificaciones rediseñada: switch general, checkboxes por canal/tipo, alertas de guardado/restablecer.
- Página /dashboard/notificaciones funciona como panel maestro-detalle con filtros por estado y tipo.

Checklist de errores relevantes (Frontend)

- Pantallas en blanco por JSX desbalanceado tras refactors → corregidos fragments y cierres.
- Layout se quedaba “colgado” cuando backend no devolvía datos → se muestra alerta y se aplican defaults.
- Botón de campana navegaba a “reportes” → ajustado para abrir lista detallada.

Pendientes sugeridos (Frontend)

- Añadir skeleton loading al modal de paneles y al detalle de notificaciones.
- Guardar filtros preferidos en reportes para pre cargar en visitas futuras.
- Confirmaciones suaves antes de restablecer (modal).

Base de datos (SQL Server)

- **Estado actual:**
 - dashboard_panel_layouts (layout por usuario/proyecto).
 - dashboard_user_format_prefs (fecha/moneda por usuario).
 - config_notificaciones (preferencias de canales y tipos).
- **Errores resueltos:** permisos de CREATE TABLE y migraciones manuales; se documentó el proceso de --fake-initial.
- **Pendiente sugerido:** script de auditoría para cambios en preferencias y monitoreo de índices cuando crezcan los registros.

Operativa (cómo levantar)

Backend

1. `python -m venv venv && venv\Scripts\activate.`

2. `pip install -r backend/requirements.txt`.
3. Copiar `backend/.env.example` → `.env`. Para pruebas locales, comentar `DATABASE_URL` (usa SQLite).
4. `python backend/manage.py migrate && python backend/manage.py runserver 8000`.

Frontend

1. `npm install`.
2. `npm run dev` → <http://localhost:5173>.

Recomendaciones finales

- Documentar scripts SQL necesarios cuando la BD “produc” no permite migraciones automáticas.
- Agregar pruebas automatizadas para `ReportExportPDFView` y para el modal de paneles (component tests).
- Mejora UX: guardar filtros favoritos, notificar en tiempo real (WebSockets o SSE) y considerar paginación en notificaciones/lists.